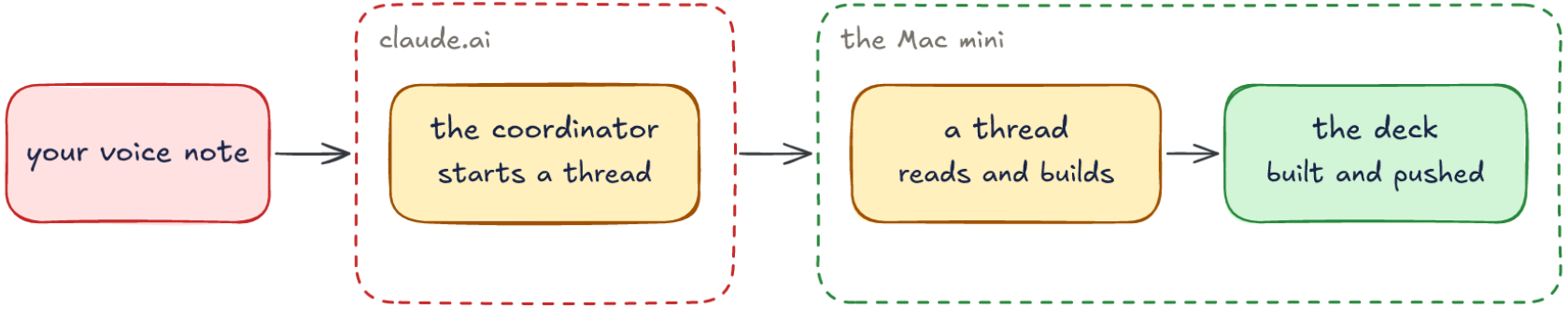
Running Claude Code Projects on Your Own Machine

EECS 498 AASE, Lecture 19

Where this lecture came from



Today's schedule

- [x] How we work
- [x] How a project works
- [ ] Break
- [x] Remote Control and rct
- [x] Projects in practice
- [ ] Questions

Part 1

How we work

Limits of one terminal session

- You have to sit at the keyboard
- One conversation holds every job
- The laptop sleeps and the work stops

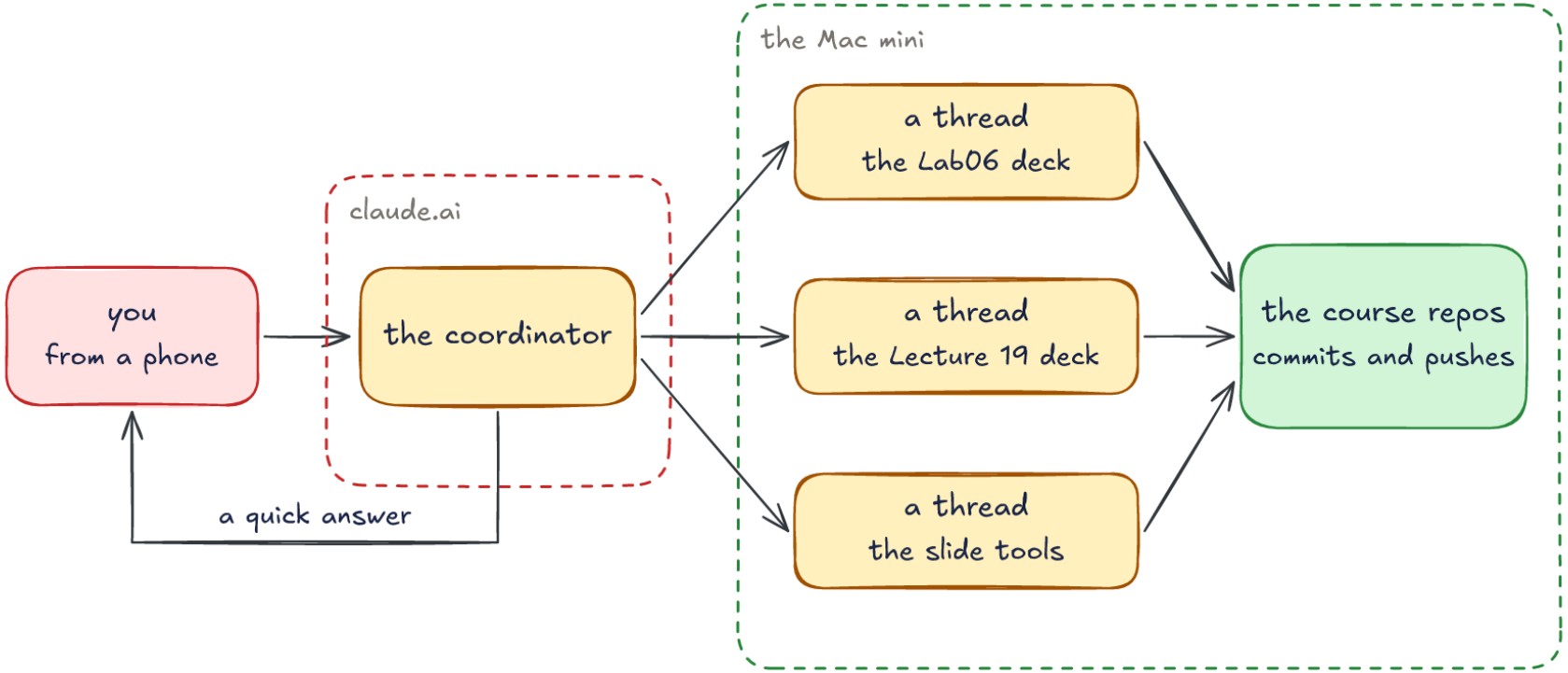
Many terminal sessions

- Every session needs its own terminal
- You brief each one from scratch
- You check each one to see what finished
- All of them run on your laptop

What we run instead

- The sessions run on a Mac mini that never sleeps
- Your phone reaches every session
- The coordinator starts sessions there for you
- Each session works in a folder we chose

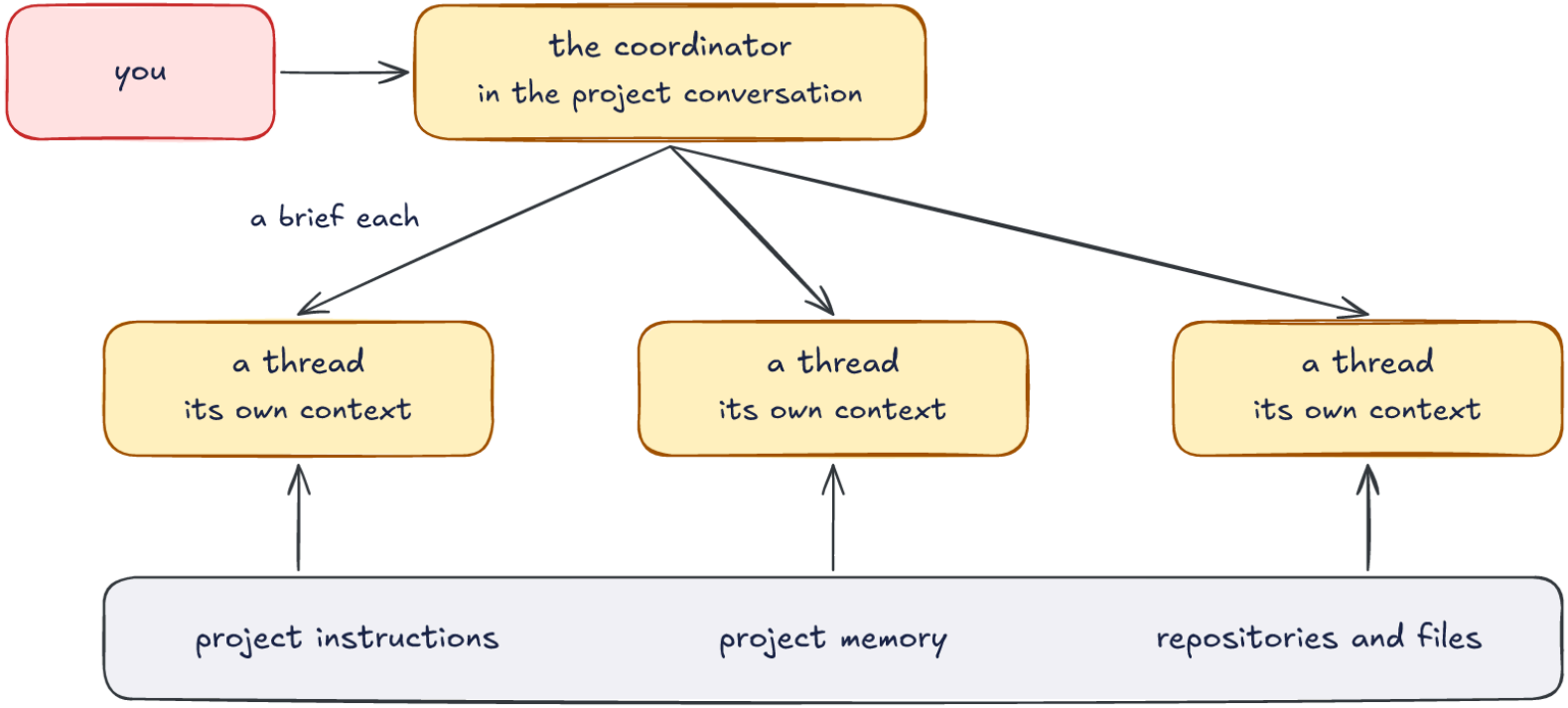
A week of course work



Part 2

How a project works

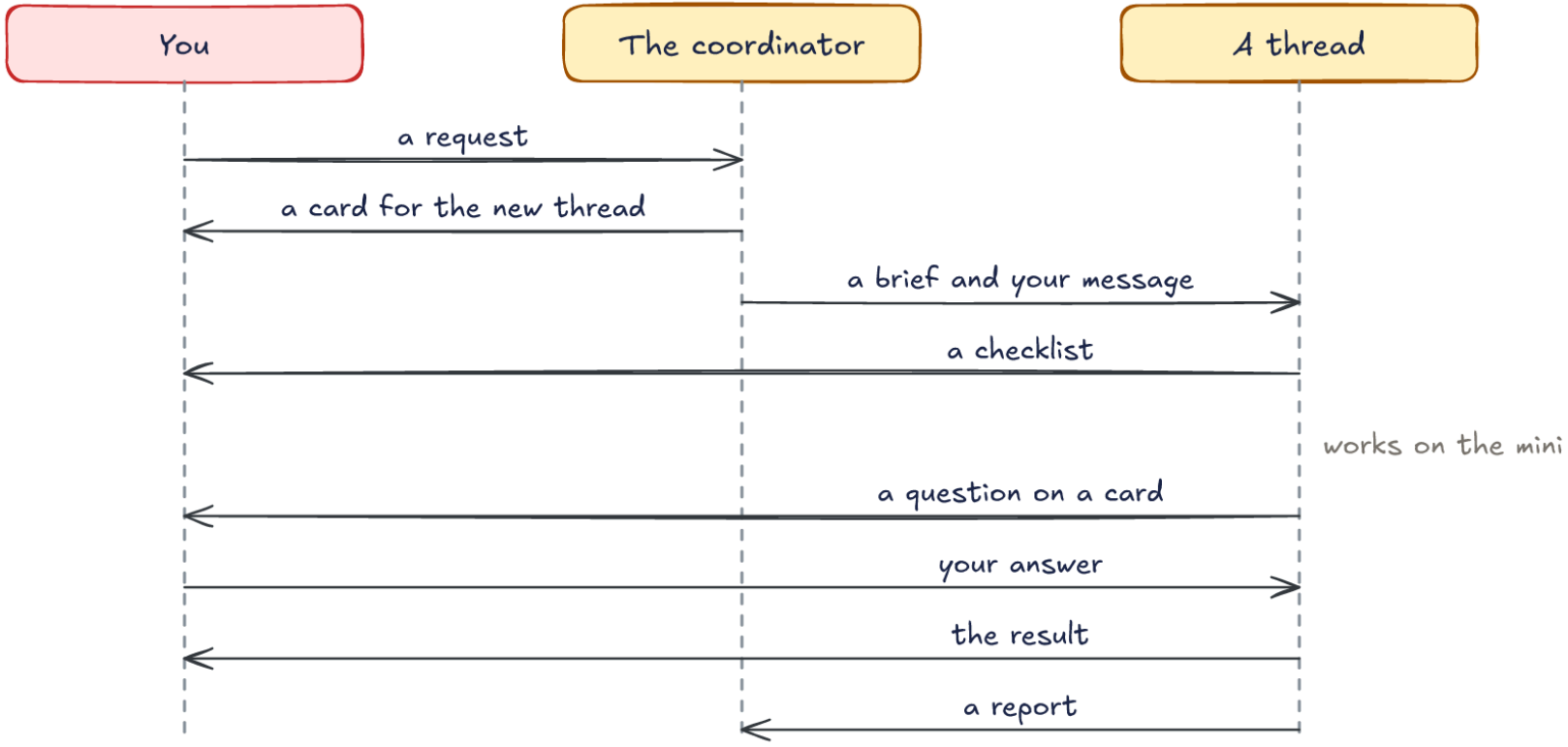
The pieces of a project



What the coordinator does

- The coordinator answers quick questions itself
- New work becomes a thread with a brief
- The coordinator sees reports, not every step
- Your preferences go into project memory

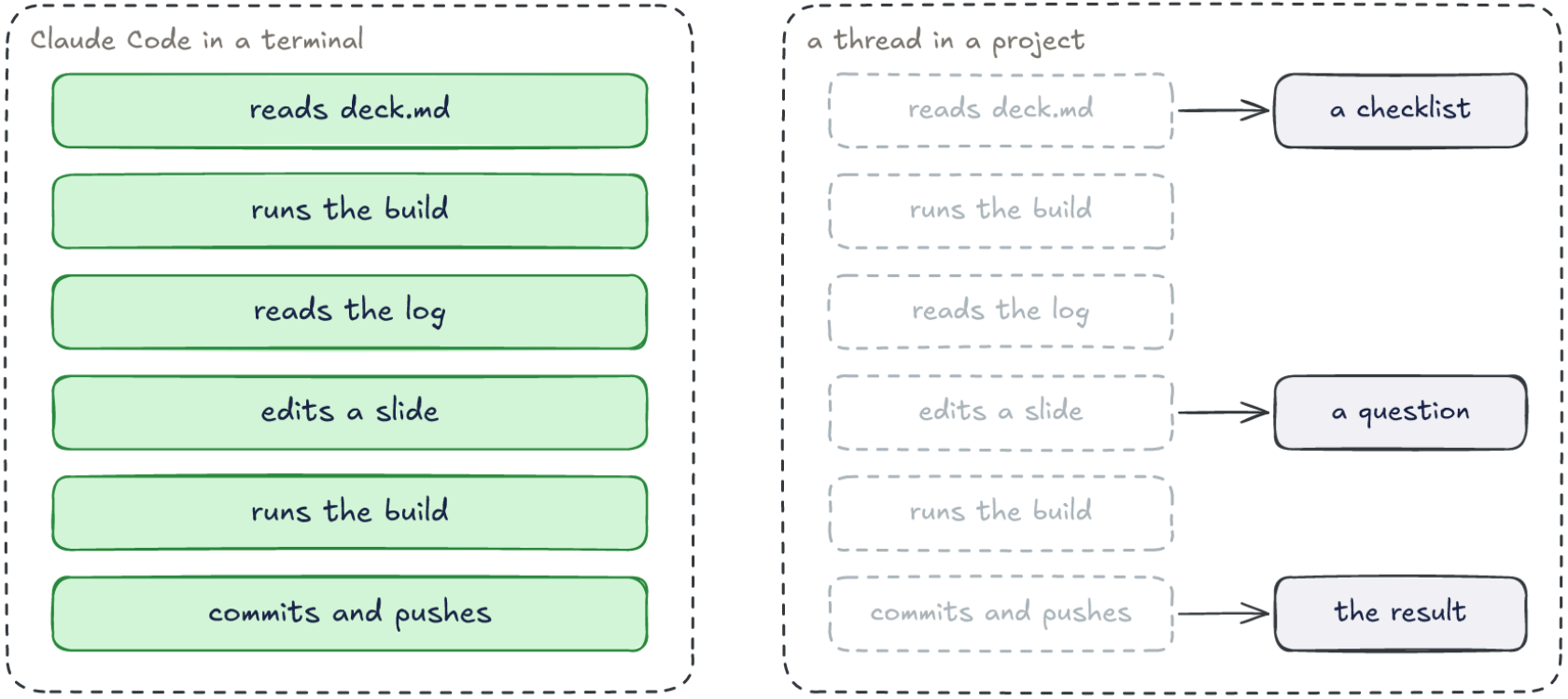
A thread's life



Live demo

A thread on the Mac mini

How much of a thread you see



The Overview pane

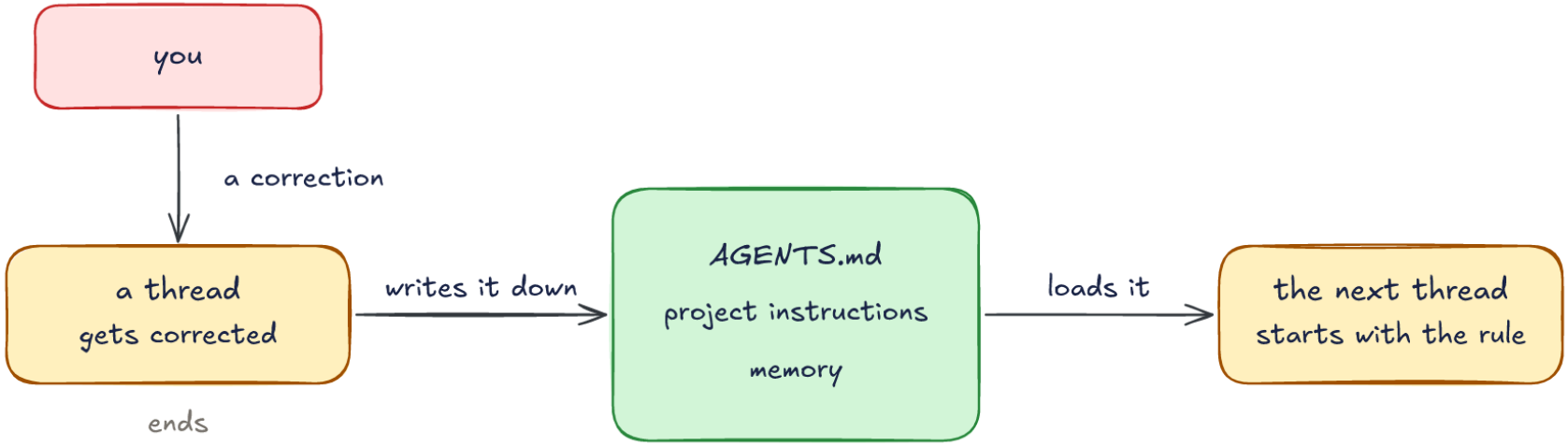
Group	What is in it
Ready for review	A pull request waits for you
Waiting on you	A reply, an approval or a failure
Working	Threads still running
Resolved	Marked done or idle for a week

Who a thread listens to

Who wrote it	How the thread treats it
You	Instructions and approval
Another Claude	A colleague's request
A file, page or comment	Data to read

Steps that can't be undone wait for you

What outlives a thread



Chat break

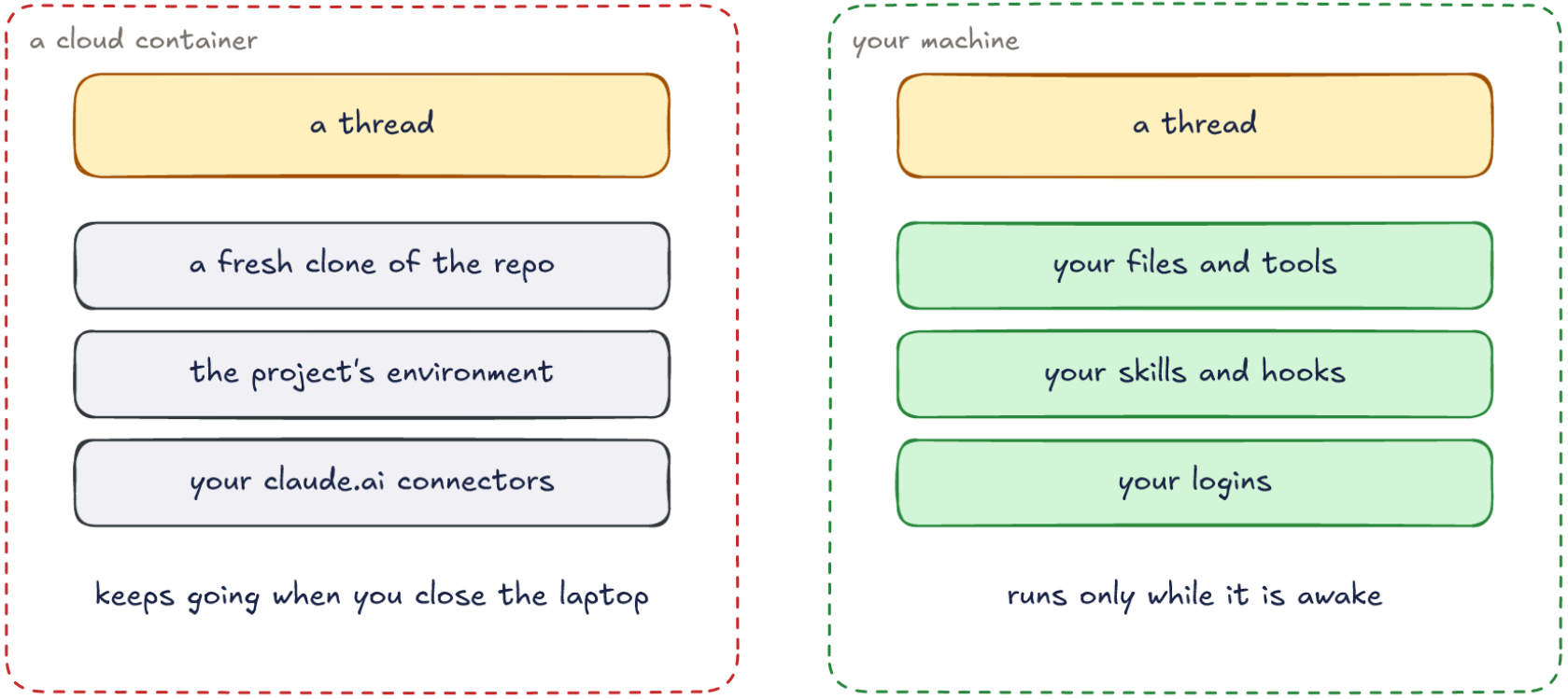
Which job in your own work would you hand to a thread?

What would that thread need from your machine?

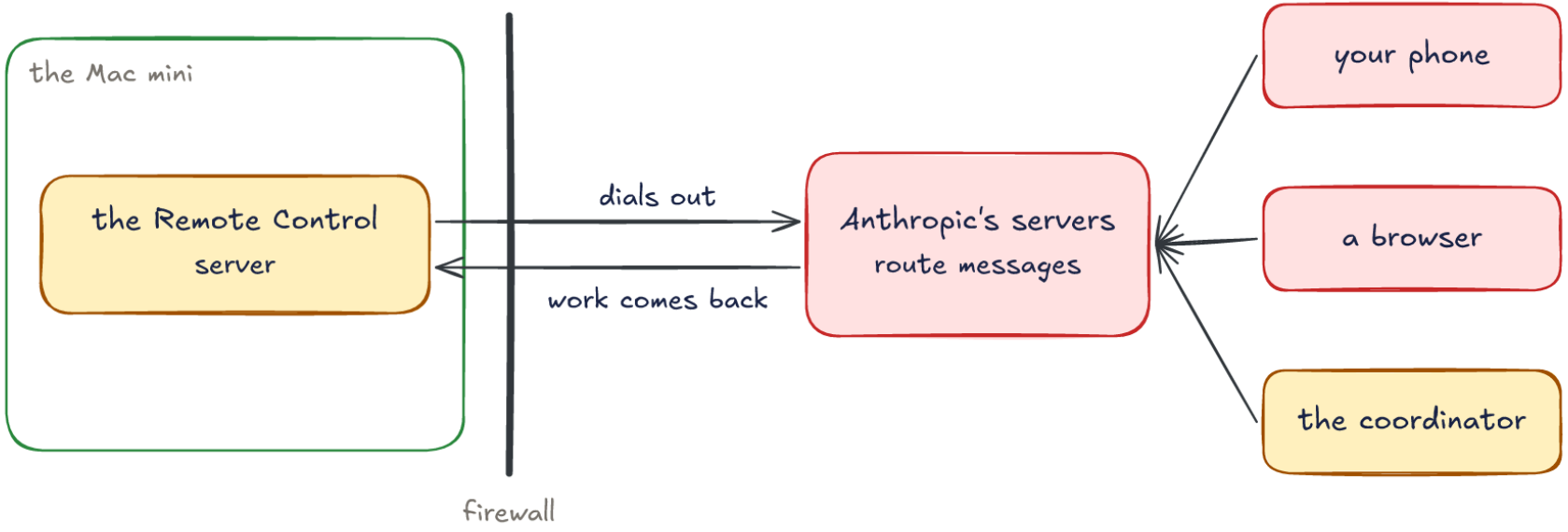
Part 3

Remote Control and rct

Where a thread runs



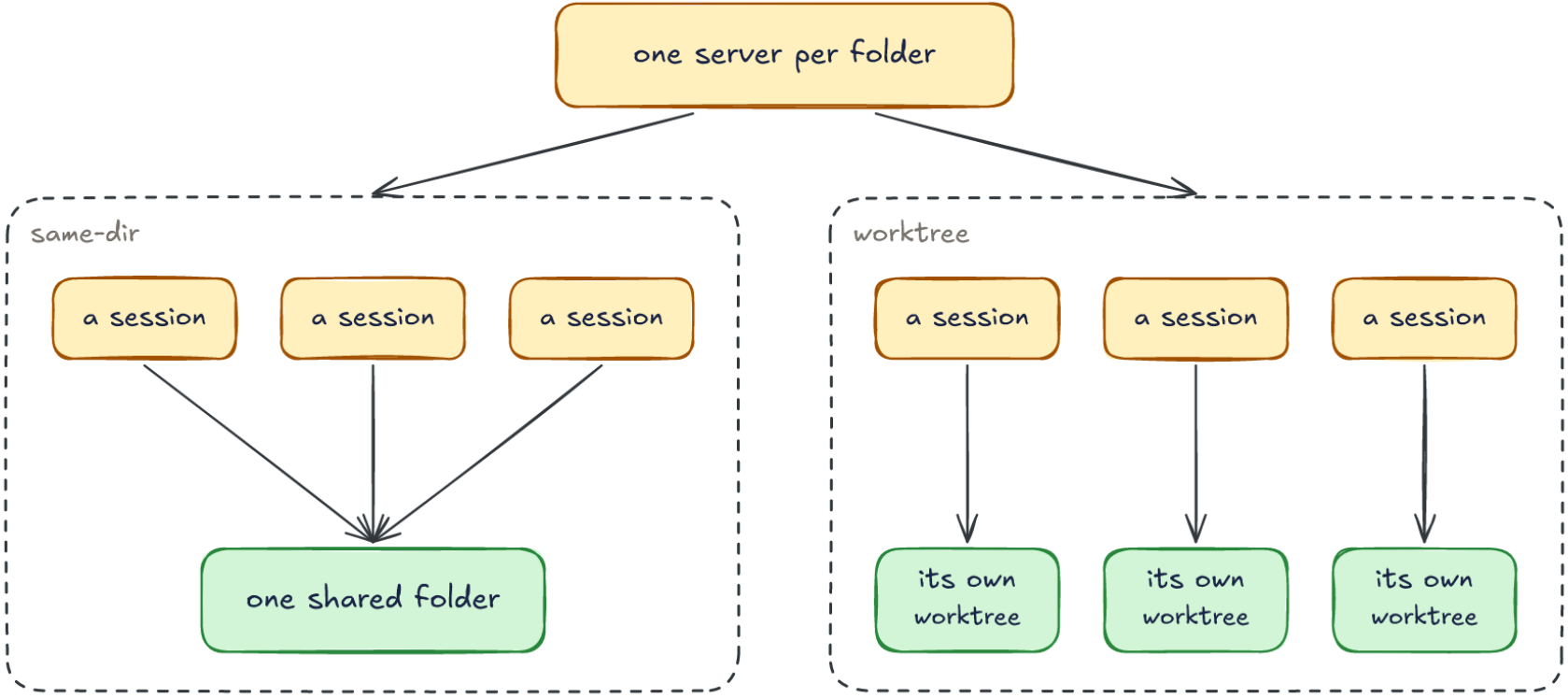
How Remote Control connects



Connecting a folder to a project

1. Run claude remote-control in the folder
2. Ask the project to work locally
3. Allow the folder on the card
4. The thread starts on your machine

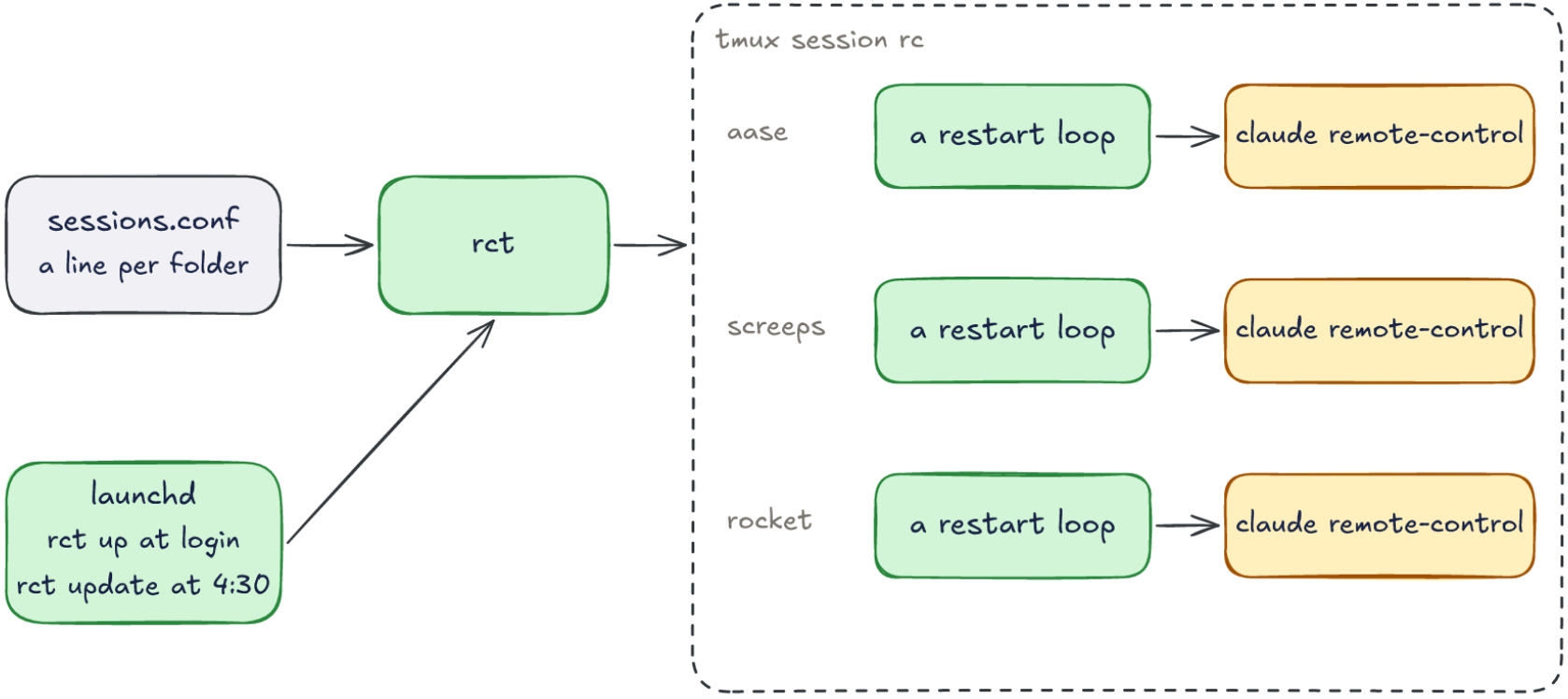
Many sessions from one server



What Remote Control leaves to you

- Something must keep the server running
- The first start asks questions
- An update needs a restart
- Each folder needs its own server

How rct works



The restart loop

```
# Remote Control needs the subscription login, not an API key
unset ANTHROPIC_API_KEY ANTHROPIC_AUTH_TOKEN ANTHROPIC_BASE_URL …
while :; do
  set -- remote-control
  # within four hours, a plain start brings the sessions back
  if [ $((now - last)) -gt "$RESUME_WINDOW" ]; then set -- "$@" --name "$name"; fi
  # nobody is there to answer "same-dir or worktree?"
  [ -n "$spawn" ] && set -- "$@" --spawn "$spawn"
  # --model never reaches spawned sessions; the env var does
  if [ -n "$model" ]; then export ANTHROPIC_MODEL="$model"; …
  claude "$@"
  …
  # died within a minute: wait twice as long, up to 5 minutes
  elif [ $(( $(date +%s) - started )) -lt 60 ]; then
    backoff=$(( backoff < 300 ? backoff * 2 : 300 ))
  …
  sleep "$backoff"
done
```

Part 4

Projects in practice

A thread's pull request

- A cloud thread works on its own branch
- A pull request opens when the work calls for it
- Auto-fix answers failing CI and comments
- Our threads push straight to main

Settings that shape a project

Setting	What it controls
Coordinator model and effort	Opus at low effort by default
Thread model and effort	Opus at high effort by default
Cloud environment	Network access, secrets and tools
Scheduled tasks	Cloud routines that run as threads

What a project draws on

- Every running thread is a full session
- The coordinator spends tokens reading reports
- A thread watching a PR wakes on CI
- An idle project costs nothing

Limits of the beta

- Projects need a Pro, Max or Ultra plan
- A project belongs to one person
- A cloud thread can lose uncommitted work
- Threads cannot move between projects

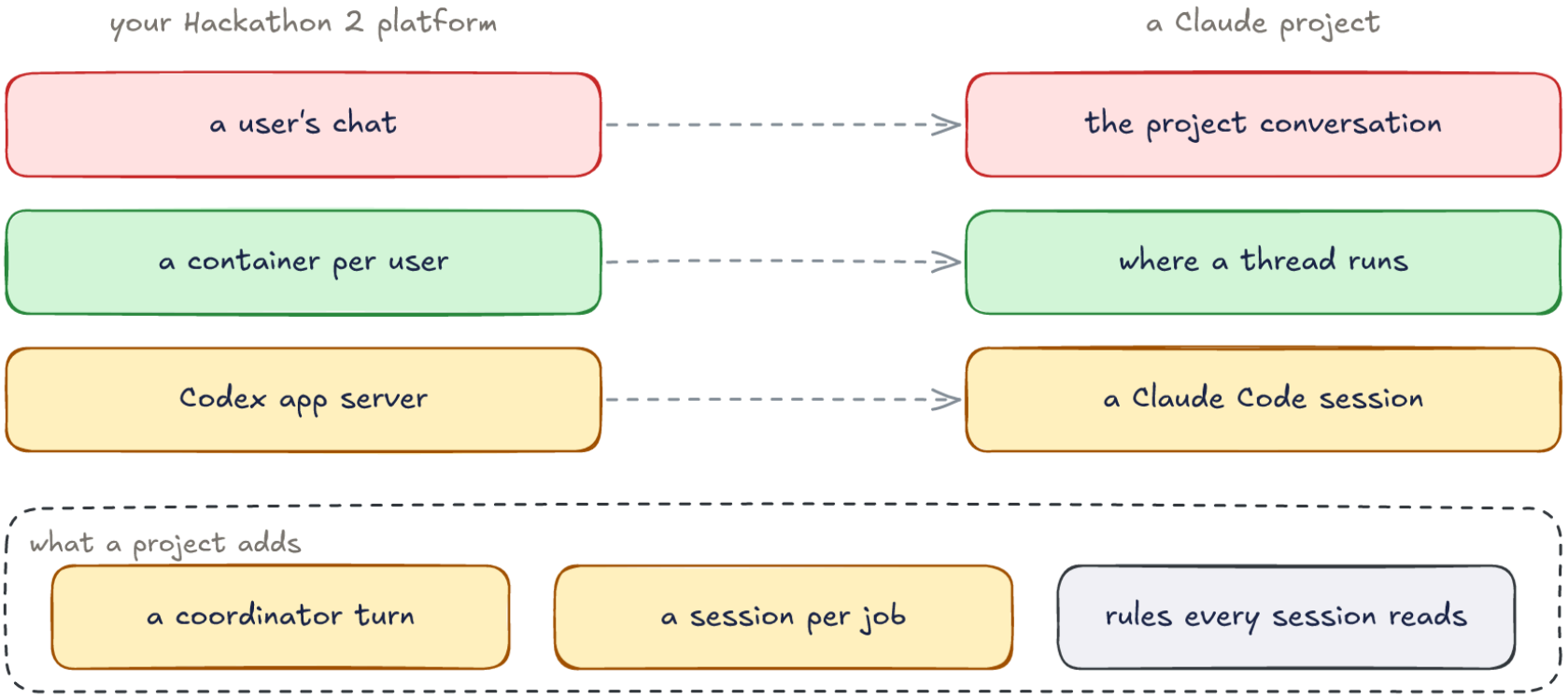
Why this works for us

- Each job gets a fresh context
- Work runs where the tools are
- The rules live in files every thread reads
- You approve only what can't be undone

Tools we built on top

Tool	What it does
`rct`	Keeps Remote Control servers running
AGENTS.md rules	Tell every thread how a deck edit ends
`bin/build-slides`	Builds and checks a deck in one command
`excalive`	Makes a deck text a thread can write

Your platform as a coordinator



Starting your own setup

1. Give each job its own session
2. Write repeated rules into AGENTS.md
3. Run sessions where your tools are
4. Automate the step after every change

Sources

Claude Code docs on Projects
code.claude.com/docs/en/claude-projects

Claude Code docs on Remote Control
code.claude.com/docs/en/remote-control

rct
github.com/RafeSymonds/rc-tmux

excalive
github.com/RafeSymonds/excalive

Questions

AASE Lecture 19, Running Claude Code Projects on Your Own Machine